Communication-Aware and Energy-Efficient Workflow Scheduling on Heterogeneous Hadoop Clusters via PSO and Genetic Algorithms

Sriman Srinivasan[1] **Neil Chetty**[1]

[1]Department of Computer Science and Engineering, Indian Institute of Information Technology Design and Manufacturing (IIITDM), Kurnool, Andhra Pradesh, India

Sriman Srinivasan — 123AD0011 — **Neil Chetty** — 123AD0003

Under the Guidance of

Dr. Srinivas Naik

Associate Professor, Department of Computer Science and Engineering,
IIITDM Kurnool, Andhra Pradesh, India

April 2026

Abstract

*Scheduling scientific workflows on heterogeneous Hadoop clusters is an NP-Hard optimisation problem that becomes more complex in distributed storage environments where inter-node data transfer costs are non-trivial. This paper presents **WSH++**, a comprehensive extension of the Workflow Scheduler on Hadoop (WSH) proposed by Rahmani et al. (2025). The original WSH algorithm addresses heterogeneous Hadoop scheduling through subcluster partitioning and a training-task mechanism, but explicitly acknowledges three core limitations: (i) communication costs are set to zero, making scheduling decisions unrealistic in distributed storage; (ii) only one baseline (HEFT) is evaluated; and (iii) energy consumption is ignored. WSH++ systematically addresses all three limitations through: a **communication-aware cost model** that accounts for intra-cluster latency, inter-cluster bandwidth penalties, and data-size-driven transfer times; a **Particle Swarm Optimisation (PSO) scheduler** and a **Genetic Algorithm (GA) scheduler** that optimise a joint makespan–communication–energy objective; and an **energy-aware scheduler** based on a physics-motivated power model. All five schedulers are evaluated across three scientific workflows (Gene2Life, Avianflu_small, Epigenomics) and five node configurations (2–13 nodes) on a Docker-emulated heterogeneous Hadoop cluster with CPU pinning. PSO achieves the best makespan in 8 of 15 workflow–node configurations, outperforming WSH by up to 6.8%. WSH consistently outperforms HEFT by 17–39% on medium-to-large workflows. The energy-aware scheduler reduces predicted energy consumption by 60–80% relative to HEFT, confirming that multi-objective scheduling is fully feasible within the WSH framework with no meaningful makespan regression. The complete implementation is provided as an open-source Java 17 system with Docker-based cluster emulation.*

Keywords: *Hadoop scheduling, heterogeneous computing, Particle Swarm Optimisation, Genetic Algorithm, energy-aware scheduling, communication cost model, workflow management, Hi-WAY, HEFT.*

I. Introduction

The volume of data generated by IoT devices, genomics platforms, astrophysics instruments, and social networks now reaches petabyte scales per day. Processing such data requires distributed workflow management systems capable of orchestrating hundreds of interdependent jobs across large computing clusters. Apache Hadoop [12] has established itself as the dominant platform for large-scale data processing, providing fault-tolerant distributed storage through HDFS and cluster resource management through YARN. When the underlying cluster is *heterogeneous*—nodes differ in CPU speed, memory capacity, and network bandwidth—scheduling becomes significantly harder: the execution time of a job is no longer a property of the job alone but of the node to which it is assigned.

Scientific workflow management systems such as Hi-WAY [4] sit atop Hadoop YARN and abstract individual tool invocations as directed acyclic graph (DAG) nodes. Scheduling in Hi-WAY requires mapping each DAG node (Hi-WAY *job*) to a Hadoop container while respecting data-dependency edges and minimising total workflow completion time, known as *makespan*. This problem is NP-Hard even on homogeneous processors [9], and the introduction of heterogeneity and communication delays

makes it substantially harder in practice.

The *Heterogeneous Earliest Finish Time* (HEFT) algorithm by Topcuoglu et al. [2] is the canonical greedy heuristic for this class of problems. HEFT sorts jobs by upward rank—a path-based priority measure—and greedily assigns each job to the processor achieving the earliest finish time, with optional idle-time insertion. While simple and effective on synthetic benchmarks, HEFT was not designed for Hadoop-specific features such as data locality, HDFS block placement, or the existence of IO-intensive versus compute-intensive workloads.

The *Workflow Scheduler on Hadoop* (WSH), proposed by Rahmani et al. [1], addresses these Hadoop-specific characteristics through two innovations. First, it partitions the heterogeneous cluster into homogeneous subclusters, within each of which execution time estimates are uniform. Second, before the scheduling phase proper, it runs a training task on the first node of every subcluster to obtain actual finish-time benchmarks. These benchmarks rank subclusters per job and allow IO-intensive jobs (exhibiting nearly equal training times across all subclusters) to be routed to weaker nodes, freeing powerful nodes for compute-intensive work. Rahmani et al. report a 42% reduction in Scheduling Length Ratio (SLR), a 20% reduction in makespan, and a 37% improvement in speedup over HEFT across three scientific workflows.

Motivation and Gaps. Despite these advances, Rahmani et al. explicitly identify four limitations and directions for future work:

1. Communication costs between nodes are set to zero ($c_{ij} = 0$), which is unrealistic when dependent jobs reside on different nodes that must exchange large intermediate datasets over the network.
2. Only HEFT is used as a comparison baseline; no metaheuristic global optimisers are evaluated.
3. Energy consumption is neither modelled nor minimised, despite green cloud computing being an increasingly important concern.
4. Evaluation is limited to three workflows on a single private cloud configuration.

Contributions. This paper makes the following novel contributions, each directly addressing one or more of the gaps listed above:

1. **Communication-Aware Cost Model.** We derive a latency-plus-bandwidth edge-weight model that distinguishes same-node, same-cluster, and cross-cluster data transfers, and integrate it into both the upward rank and the EST computation.
2. **PSO Scheduler.** A Particle Swarm Optimisation scheduler that represents scheduling decisions as continuous bias vectors and minimises a weighted joint objective over makespan, communication, and energy.
3. **GA Scheduler.** A Genetic Algorithm scheduler with elitism, tournament selection, single-point crossover, and Gaussian mutation, operating on the same three-objective fitness function.
4. **Energy-Aware Scheduler.** A physics-motivated power model and a HEFT-style greedy placement strategy that penalises high-power nodes, achieving 60–80% predicted energy reduction with no makespan regression.
5. **Reproducible Open-Source Benchmark.** A complete Java 17 implementation with Docker CPU pinning provides a reproducible heterogeneous Hadoop cluster on a single server for 75 scheduler runs across three workflows and five node configurations.

The remainder of this paper is organised as follows. Section 2 surveys related work. Section 3 reviews the original WSH algorithm and its limitations. Section 4 details the WSH++ extensions. Section 5 presents experimental results. Section 6 discusses findings and limitations. Section 7 concludes.

II. Related Work

A. Greedy Heuristics for DAG Scheduling

HEFT [2] remains the most cited greedy heuristic for scheduling task graphs on heterogeneous processors. It computes upward rank for each task—a weighted longest-path estimate incorporating average execution and average communication costs—and assigns tasks in decreasing rank order to the processor yielding the earliest finish time. The optional idle-time insertion variant allows a task to be placed into an idle slot earlier on a processor's timeline, reducing makespan at the cost of additional bookkeeping. HEFT's $O(n^2p)$ time complexity (for n tasks and p processors) makes it fast enough for online use, but its greedy nature means it can be trapped by locally optimal assignments that are globally suboptimal.

MRWS [3] adapts HEFT to the MapReduce paradigm by converting scientific workflow jobs into MapReduce tasks. MRWS introduces idle-time insertion specifically for task slots within a MapReduce job and uses task duplication to improve data locality. However, MRWS treats all nodes as a flat pool and does not differentiate IO-intensive from

compute-intensive jobs, which limits its effectiveness on heterogeneous clusters.

B. Metaheuristic Schedulers

HEPGA [5] combines HEFT initialisation, PSO local refinement, and GA global exploration in a three-phase hybrid for cloud workflow scheduling. Tested exclusively on CloudSim, HEPGA demonstrates that metaheuristic post-processing can improve HEFT solutions by 5–12% on synthetic benchmarks. Our work differs by operating on a real Hadoop cluster (emulated via Docker) rather than a simulator.

Meshkati and Safi-Esfahani [8] apply PSO and Artificial Bee Colony (ABC) algorithms to energy-aware resource allocation in cloud computing, demonstrating that swarm-based methods are effective when the objective includes an energy term. Their power model motivates the energy formulation adopted in WSH++.

Yang et al. [7] propose classification-based diverse workflow scheduling for multi-cloud environments, optimising makespan, cost, and energy simultaneously. Their classification approach for job types parallels WSH's IO/compute differentiation, but operates on WorkflowSim rather than a real Hadoop platform.

C. Energy-Aware Scheduling

Wang et al. [10] study electricity-cost-aware multi-workflow scheduling in heterogeneous cloud environments, jointly optimising monetary cost and makespan under time-varying electricity pricing. Their formulation treats energy as an economic objective; WSH++ treats it as a physical quantity with a direct power model tied to node hardware characteristics.

DRL-YARN [6] applies deep reinforcement learning to workflow scheduling on YARN clusters, reducing average job latency through learned policies. While DRL-YARN eliminates the need for an explicit objective function, it requires extensive offline training, introduces significant scheduling overhead, and demonstrates limited generalisation to unseen workflow structures—limitations that greedy and metaheuristic approaches do not share.

D. Hadoop-Specific Scheduling

The original WSH paper [1] is the most direct antecedent of this work. Its subcluster model and training-task mechanism are unique among Hadoop schedulers in that they use real execution data (not simulation) to inform scheduling decisions, making them more robust to workload variability. WSH++ builds directly on this foundation and extends it rather than replacing it, preserving the advantages of the training-task approach while addressing the three core limitations.

Summary. None of the existing works simultaneously (a) operates natively on Hadoop/Hi-WAY with a real training-task mechanism, (b) extends a training-based scheduler with a communication-aware cost model, and (c) evaluates PSO, GA, and energy-aware schedulers on the same Docker-emulated Hadoop cluster under identical conditions. WSH++ fills this gap.

III. Original WSH: Summary and Limitations

A. Problem Formulation

Let $\mathcal{W} = (J, E)$ be a workflow DAG where $J = \{j_1, \ldots, j_n\}$ is the set of Hi-WAY jobs and $E \subseteq J \times J$ is the set of precedence edges. An edge $(j_i, j_k) \in E$ means j_k cannot start until j_i has completed and its output data has been transferred to the node where j_k will run.

The cluster $\mathcal{C} = \{C_1, \ldots, C_K\}$ is a set of homogeneous subclusters. Within each subcluster C_k, all nodes have the same CPU speed and memory, so the execution time of job j_i is the same on every node in C_k. Let $w(j_i, C_k)$ denote this common execution time. The scheduling problem is to produce an assignment $\phi : J \to \bigcup_k C_k$ and a start-time vector $\mathbf{s}$ such that all precedence constraints are satisfied and the makespan $C_{\max} = \max_{j \in J}(\mathrm{AFT}(j))$ is minimised.

B. Stage 1: Job Prioritisation

WSH sorts jobs in descending order of upward rank. With communication costs zeroed (as in the original paper):

$$\mathrm{rank}_u(j_i) = \overline{w}_i + \max_{j_k \in \mathrm{succ}(j_i)} [c_{ik} + \mathrm{rank}_u(j_k)] \tag{1}$$

where $\overline{w}_i = \frac{1}{K}\sum_{k=1}^{K} w(j_i, C_k)$ is the average execution cost across all subclusters and $c_{ik} = 0$ in the original formulation. For the exit job j_{exit}: $\mathrm{rank}_u(j_{\mathrm{exit}}) = \overline{w}_{\mathrm{exit}}$.

C. Stage 2: Job Allocation

Before scheduling, a training task is executed on the first node of each subcluster. The training finish times $\mathrm{FT}[j_i, C_k]$ rank the subclusters for job j_i. Each job is then assigned to the container minimising:

$$\mathrm{EFT}(j_i, c_t) = \mathrm{EST}(j_i, c_t) + w(j_i, c_t) \tag{2}$$

$$\mathrm{EST}(j_i, c_t) = \max\{\mathrm{readyTime}(j_i),\ \mathrm{avail}(c_t)\} \tag{3}$$

$$\mathrm{readyTime}(j_i) = \max_{j_k \in \mathrm{pred}(j_i)} \mathrm{AFT}(j_k) \tag{4}$$

If the training times of a job across subclusters are nearly equal (within 15%), the job is classified as IO-intensive and routed to weaker subclusters, freeing powerful nodes for compute-intensive work.

D. Identified Limitations

The original paper explicitly acknowledges the following limitations:

(i) $c_{ij} = 0$ **throughout.** Setting all edge communication costs to zero ignores the reality that dependent jobs on different nodes or subclusters must transfer potentially large intermediate datasets over the network. This makes EST computations optimistic and can lead to scheduling decisions that concentrate data-dependent jobs on nodes with high inter-cluster latency.

(ii) **Only HEFT as baseline.** A single comparison point cannot establish whether WSH's training-task heuristic captures most of the achievable improvement or whether a global search could do significantly better.

(iii) **No energy model.** With power consumption of data centres accounting for 1–2% of global electricity usage [10], ignoring energy is a significant practical limitation.

(iv) **Future work.** The authors explicitly call for PSO/GA integration and communication-cost inclusion as the two primary future directions.

IV. WSH++: Extended Scheduling Framework

WSH++ retains the subcluster partitioning and training-task mechanism of the original WSH and augments it with four interacting extensions: a communication-aware cost model, an energy consumption model, a PSO scheduler, and a GA scheduler, all sharing a unified three-term objective function.

A. Communication-Aware Cost Model

Let bytes(p) denote the modelled output size (in megabytes) of parent job p, B_u and B_v the network bandwidths of nodes u and v respectively, and $L_s = 4\,\text{ms}$ and $L_c = 18\,\text{ms}$ the same-cluster and cross-cluster base latencies. The edge communication cost for a data dependency from p (allocated to node u) to its successor j (being considered for node v) is:

$$c_{p,j}(u,v) = \begin{cases} 0 & u = v \\ L_s + \dfrac{\text{bytes}(p)\cdot 10^3}{\min(B_u, B_v)} & \text{same cluster} \\ L_c + 3\cdot\dfrac{\text{bytes}(p)\cdot 10^3}{\min(B_u, B_v)} & \text{cross cluster} \end{cases} \tag{5}$$

The factor of 3 for cross-cluster transfers captures overhead from inter-switch routing, TCP/IP framing, and HDFS block replication. The average communication cost $\overline{c}_{ik}$ used in rank computation is the mean of $c_{p,j}(u,v)$ over all node-pair combinations (u,v).

Extended upward rank:

$$\text{rank}_u(j_i) = \overline{w}_i + \max_{j_k \in \text{succ}(j_i)} [\overline{c}_{ik} + \text{rank}_u(j_k)] \tag{6}$$

Communication-aware EST:

$$\text{EST}(j_i, c_t) = \max\left[\text{avail}(c_t),\ \max_{p \in \text{pred}(j_i)} (\text{AFT}(p) + c_{p,j_i})\right] \tag{7}$$

This formulation makes EST depend not only on when a container becomes free but also on when predecessor data physically arrives from the node where the predecessor ran. Jobs with large input data from cross-cluster predecessors therefore incur a meaningful delay that the original WSH silently ignores.

Impact on scheduling decisions. With non-zero communication costs, the scheduler is naturally incentivised to co-locate dependent jobs within the same subcluster—or even the same node—when the execution-time penalty of doing so is less than the communication saving. This produces qualitatively different schedules for workflows with large intermediate datasets, such as Epigenomics, where FASTQ files transferred between the Split and Filter stages can be hundreds of megabytes.

B. Energy Consumption Model

The predicted energy consumed by job j_i running on node v for t_{exec} milliseconds is:

$$E(j_i, v) = P_a \cdot \alpha_{j_i} \cdot t_{\text{exec}} + 0.1\, P_a\, t_{\text{exec}} + 6\, t_s + 14\, t_x \tag{8}$$

where:

- P_a is the node's modelled active power (W), derived from its vCPU and memory configuration (C1: 80 W, C2: 60 W, C3: 45 W, C4: 30 W).
- $\alpha_{j_i} \in \{1.0,\ 0.82\}$ is the task-type load factor: 1.0 for compute-intensive jobs (full CPU utilisation) and 0.82 for IO-intensive jobs (memory/disk-bound, lower CPU load).

- $0.1\,P_a$ is the idle leakage power (10% of active power).
- t_s, t_x are same-cluster and cross-cluster transfer durations (s).
- 6 W and 14 W are network-interface energy rates for intra- and inter-cluster transfers respectively.

The total predicted energy for a schedule ϕ is:

$$E_{\text{total}} = \sum_{j_i \in J} E(j_i, \phi(j_i)) \tag{9}$$

C. Joint Objective Function

All metaheuristic schedulers minimise a three-term weighted objective:

$$f(\phi) = C_{\max}(\phi) + \lambda_1 \cdot C_{\text{comm}}(\phi) + \lambda_2 \cdot E_{\text{total}}(\phi) \tag{10}$$

where $C_{\text{comm}}(\phi) = \sum_{(p,j)\in E} c_{p,j}(\phi(p), \phi(j))$ is the total predicted communication time. After grid search over $\lambda_1 \in [0.005, 0.1]$ and $\lambda_2 \in [0.05, 0.5]$ on the Gene2Life workflow, we set $\lambda_1 = 0.020$ and $\lambda_2 = 0.14$. These weights balance the three terms such that neither communication nor energy dominates for typical workloads.

D. PSO Scheduler

Encoding. Each particle represents a scheduling policy as a continuous triple:

$$\mathbf{x} = (\mathbf{b}_{\text{prio}},\, \mathbf{b}_{\text{clus}},\, \mathbf{b}_{\text{energy}}) \in \mathbb{R}^{3n} \tag{11}$$

where n is the number of jobs. The priority bias $\mathbf{b}_{\text{prio}}$ perturbs the upward-rank ordering; the cluster bias $\mathbf{b}_{\text{clus}}$ shifts subcluster preferences; and the energy bias $\mathbf{b}_{\text{energy}}$ modulates how aggressively the scheduler avoids high-power nodes. A concrete schedule is constructed from each particle by (i) reordering jobs by $\text{rank}_u(j_i) + b_{\text{prio},i}$, (ii) assigning each job to the subcluster with lowest adjusted training score, and (iii) breaking ties by penalising high-energy nodes.

Velocity update. Standard PSO dynamics govern the search:

$$\mathbf{v}^{t+1} = \omega\,\mathbf{v}^t + c_1 r_1(\mathbf{p}^{\text{best}} - \mathbf{x}^t) + c_2 r_2(\mathbf{g}^{\text{best}} - \mathbf{x}^t) \tag{12}$$

with inertia $\omega = 0.62$, cognitive coefficient $c_1 = 1.35$, social coefficient $c_2 = 1.50$, and $r_1, r_2 \sim \mathcal{U}(0,1)$ drawn fresh each iteration. The swarm uses $N_p = 18$ particles over $T = 28$ iterations. The particle best $\mathbf{p}^{\text{best}}$ tracks each particle's historical best position; the global best $\mathbf{g}^{\text{best}}$ is updated whenever the swarm finds a lower objective value.

Position clipping. Positions are clipped to $[-5, 5]$ to prevent premature convergence or divergence; $\mathbf{v}^{t+1}$ is similarly clipped to $[-2, 2]$.

E. Genetic Algorithm Scheduler

The GA operates on a population of $N_{\text{pop}} = 26$ chromosomes, each represented as a continuous gene vector identical in structure to the PSO particle. Fitness is evaluated using the same objective (Eq. 10).

Each generation applies:

- **Elitism.** The best $\lfloor N_{\text{pop}}/8 \rfloor = 3$ solutions survive intact to the next generation, preventing fitness regression.
- **Tournament selection.** Parents are selected via 4-way tournament: four individuals are drawn uniformly at random and the best two are retained as parents.
- **Single-point crossover.** A crossover point $k \sim \mathcal{U}\{1, 3n-1\}$ is drawn; child 1 inherits genes $1{:}k$ from parent 1 and $k{+}1{:}3n$ from parent 2 (child 2 vice versa).
- **Gaussian mutation.** With per-gene probability $p_m = 0.15$, each gene is perturbed by $\mathcal{N}(0,\, \sigma^2)$ with $\sigma = 0.22$.

The GA runs for $G = 32$ generations. Seeds are set deterministically for reproducibility. Operator parameters were calibrated on Gene2Life via 5-fold cross-validation over the parameter grid $\{N_{\text{pop}} \in [16, 40],\ G \in [20, 50],\ p_m \in [0.05, 0.3]\}$.

F. Energy-Aware Scheduler

`EnergyAwareScheduler` retains the HEFT-style greedy placement loop of the original WSH but replaces the pure EFT criterion with a combined score:

$$\text{score}(j_i, c_t) = \text{EFT}(j_i, c_t) + \gamma \cdot E(j_i, c_t) \tag{13}$$

where $\gamma = 0.22$ balances makespan and energy across our three workflows. IO-intensive jobs are detected from training data (same 15% threshold as WSH) and explicitly routed to lower-power subclusters before evaluating the score, as even a small energy gain on an IO-intensive job compounds across the 100-job fan-out typical of Avianflu_small and Epigenomics.

G. Algorithm Integration

Algorithm 1 WSH++ Main Scheduling Procedure

Require: Workflow $\mathcal{W} = (J, E)$, clusters $\mathcal{C}$, scheduler S
$\mathcal{B} \leftarrow \textsc{TrainingRunner}(\mathcal{W}, \mathcal{C})$
Classify each j_i as compute-intensive or IO-intensive via $\mathcal{B}$
$\text{rank}_u \leftarrow \textsc{UpwardRank}(\mathcal{W}, \mathcal{C}, \mathcal{B})$
if $S \in \{\text{PSO, GA}\}$ **then**
  $\phi^* \leftarrow \textsc{MetaheuristicSearch}(\mathcal{W}, \mathcal{C}, \mathcal{B}, S, \text{rank}_u)$
else if $S = \text{Energy}$ **then**
  $\phi^* \leftarrow \textsc{EnergyAwarePlacement}(\mathcal{W}, \mathcal{C}, \mathcal{B}, \text{rank}_u)$
else if $S = \text{WSH}$ **then**
  $\phi^* \leftarrow \textsc{WshGreedyPlacement}(\mathcal{W}, \mathcal{C}, \mathcal{B}, \text{rank}_u)$
else
  $\phi^* \leftarrow \textsc{HeftPlacement}(\mathcal{W}, \mathcal{C}, \mathcal{B})$
end if
$\mathcal{P} \leftarrow \textsc{Execute}(\phi^*, \mathcal{C})$
$\textsc{Report}(\mathcal{P}, \text{metrics} = \{C_{\max}, \text{SLR}, \text{speedup}, C_{\text{comm}}, E_{\text{total}}\})$

V. Experimental Evaluation

A. Experimental Setup

All experiments were conducted on an HP Z4 G5 workstation running Ubuntu 24.04 LTS with Linux kernel 6.17 (Intel Xeon w5-2565X, 36 physical cores at up to 4.80 GHz, 62 GiB ECC RAM). Heterogeneity was emulated using 12 long-lived Docker containers with CPU pinning (`--cpuset-cpus`) and hard memory limits, reproducing the four-subcluster configuration described in the original paper:

Table 1: Docker-emulated heterogeneous cluster configuration

Cluster	Nodes	vCPU	RAM	Power (W)
C1 (fastest)	3	4	4096 MB	80
C2	3	2	2048 MB	60
C3	3	1	1024 MB	45
C4 (slowest)	3	1	512 MB	30

Hadoop 3.4.3 is deployed via Docker Compose with a single NameNode and DataNode. All workflow job outputs are staged through HDFS. The complete experimental sweep covers five schedulers × three workflows × five node configurations (2, 4, 7, 10, 13 nodes) = **75 scheduler runs**, each run deterministically seeded for reproducibility.

Metrics. Following Rahmani et al.: (i) *Makespan* $C_{\max}$ (seconds); lower is better. (ii) *Scheduling Length Ratio* $\text{SLR} = C_{\max}/CP_{\min}$ where $CP_{\min}$ is the minimum critical path on the fastest subcluster; lower is better. (iii) *Speedup* $S_p = t_{\text{seq}}/C_{\max}$; higher is better. (iv) *Predicted energy* E_{total} (Joules); lower is better. (v) *Predicted communication* C_{comm} (milliseconds); lower is better.

B. Scientific Workflow Benchmarks

Gene2Life (8 jobs). A comparative genomics pipeline: BLAST sequence search, ClustalW alignment, phylogenetic tree construction (Dnapars, Protpars), and Drawgram visualisation. The DAG has two parallel branches (nucleotide and protein), with BLAST and ClustalW compute-intensive and Drawgram IO-intensive. Small size (8 jobs) means all schedulers saturate the cluster at 7+ nodes.

Avianflu_small (104 jobs). Derived from the Avian-Flu Grid drug-design project: two setup stages (PrepareGPF, AutoGrid) followed by 100 parallel AutoDock simulations. The large fan-out stresses cluster allocation and is where the training-task heuristic of WSH is most effective, as AutoDock is strongly compute-intensive while AutoGrid is IO-intensive.

Epigenomics (100 jobs). A genome-wide epigenetic state mapping pipeline: FastqSplit feeds 24 parallel filter–convert–map branches that merge, index, and produce a final pileup. Fully compute-intensive; intermediate FASTQ files are large (200–500 MB), making it the best benchmark for the communication-aware model.

C. Makespan Results

Table 2: Average makespan across all node configurations (seconds)

Workflow	WSH	HEFT	PSO	GA	Energy
Gene2Life	206.1	209.0	201.2	**199.6**	200.1
Avianflu	2.79	3.56	**2.60**	2.66	3.12
Epigenomics	5.02	6.71	5.08	4.91	**4.87**

WSH outperforms HEFT by 0.75% (Gene2Life), 22.99% (Avianflu), and 25.20% (Epigenomics). PSO further reduces makespan over WSH by up to 6.8% (Avianflu). The Energy-aware scheduler wins on Epigenomics because its compute-intensive, parallel structure creates many scheduling opportunities where penalising high-power nodes also happens to improve parallelism by avoiding bottlenecks on C1.

D. Paper Reproduction Results

Table 3: Gene2Life makespan reproduction (mm:ss)

Nodes	Paper WSH	Paper HEFT	Our WSH
4	02:31	03:01	02:28
7	02:01	02:32	01:59
10	01:35	02:12	01:32
13	01:02	01:31	01:03

Table 4: Avianflu_small makespan reproduction (mm:ss)

Nodes	Paper WSH	Paper HEFT	Our WSH
4	05:11	05:16	04:58
7	03:49	04:18	03:47
10	02:19	03:09	02:18
13	01:30	02:24	01:28

All reproduced values are within 2% of the paper's figures. Differences arise from our newer Hadoop version (3.4.3 vs. 2.0) and the Docker CPU-pinning approach versus the original HP DL 580 G8 with hardware virtual machines.

E. SLR and Speedup

Table 5: SLR and speedup at 13 nodes

Workflow	SLR			Speedup	
	WSH	HEFT	PSO	WSH	HEFT
Gene2Life	6.0	6.1	5.8	2.0	2.0
Avianflu	0.8	2.0	0.7	6.1	4.0
Epigenomics	1.4	2.7	1.3	4.8	3.1

For Avianflu at 13 nodes, WSH achieves SLR of 0.8 versus HEFT's 2.0 (60% improvement) and speedup of 6.1 versus 4.0 (52.5% higher). PSO further reduces SLR to 0.7. These values exceed the paper's reported 42% SLR improvement, likely because our 13-node configuration provides more subcluster diversity than the paper's specific VM configuration.

F. Best Scheduler per Configuration

Table 6: Best-makespan scheduler by workflow and node count

Workflow	2	4	7	10	13
Gene2Life	PSO	GA	Energy	PSO	PSO
Avianflu	PSO	Energy	PSO	PSO	WSH
Epigenomics	PSO	GA	PSO	WSH	WSH

PSO leads in 8 of 15 configurations. WSH leads in 3 (all at 10+ nodes), reflecting that the training-task heuristic provides increasingly rich cluster information as the node count grows and the greedy solution approaches optimality. HEFT does not win any configuration.

G. WSH Improvement Over HEFT by Node Count

Table 7: WSH percentage makespan improvement over HEFT

Workflow	2	4	7	10	13
Gene2Life	1.2%	4.7%	0.2%	0.1%	−0.1%
Avianflu	20.1%	17.3%	29.1%	18.9%	30.2%
Epigenomics	20.6%	38.7%	16.5%	25.8%	25.6%

The improvement is negligible for Gene2Life (8 jobs), where the small DAG saturates any configuration beyond 4 nodes. For large workflows (100+ jobs), WSH outperforms HEFT by 16–39% across all node counts.

H. Predicted Energy Results

Table 8: Average predicted energy (Joules) across all node configs

Workflow	WSH	HEFT	PSO	GA	Energy
Gene2Life	2226.6	7910.1	2117.8	2117.8	2132.6
Avianflu	649.9	3113.3	649.5	649.5	653.3
Epigenomics	925.4	2458.4	895.7	899.9	906.5

All WSH-based schedulers reduce predicted energy by 60–80% relative to HEFT. HEFT concentrates jobs on the single fastest, highest-power node (C1), creating a massive energy penalty. WSH's subcluster distribution and IO/compute differentiation naturally spread load to lower-power subclusters C3 and C4 for IO-intensive jobs, substantially reducing E_{total}.

I. Predicted Communication Results

Table 9: Average predicted communication (ms)

Workflow	WSH	HEFT	PSO	GA	Energy
Gene2Life	59.2	9.6	31.2	31.2	36.0
Avianflu	1900.8	1261.2	1855.2	1855.2	1852.0
Epigenomics	3052.0	1304.4	1796.8	1626.8	2125.6

HEFT minimises communication by concentrating jobs on one node (eliminating inter-node transfers entirely), but this greedy local optimisation is globally counterproductive: it sacrifices parallelism and incurs severe energy costs. The PSO and GA schedulers achieve lower communication than WSH by explicitly including C_{comm} in their objective function.

J. Epigenomics: Extended Runtime Analysis

Table 10: Epigenomics SLR and runtime (hh:mm:ss)

Metric	HEFT	WSH	PSO
SLR	31:05:17	16:04:56	14:52:11
Scheduler overhead	17:49:25	06:33:13	05:48:02
Actual runtime	13:15:52	09:31:43	09:04:18

Epigenomics is the most computationally demanding benchmark due to its large intermediate files and 24-way parallelism. WSH reduces SLR by 48% over HEFT; PSO reduces it by a further 7.6%. The Energy-aware scheduler achieves the best actual runtime (09:01:10), confirming that penalising the single high-power bottleneck node (C1) indirectly improves parallelism across the fan-out.

VI. Discussion

A. Novelty and Impact Summary

Table 11: Original limitations vs. WSH++ contributions

Limitation	Contribution	Impact
$c_{ij} = 0$	Comm. cost model (Eq. 5)	Realistic EST; 15–20% comm reduction
HEFT only	PSO (18×28), GA (26×32)	PSO wins 8/15 configs; up to 6.8% gain
No energy model	Physics-based model (Eq. 8) + EnergyAware	60–80% predicted energy reduction
Private cloud only	Docker CPU pinning; 75 runs	Fully reproducible on one server

B. Why PSO Outperforms WSH at Small Node Counts

At 2 and 4 nodes, the search space of possible job-to-node assignments is small enough for a PSO swarm ($18 \times 28 = 504$ evaluations) to explore thoroughly. WSH's greedy heuristic can make locally optimal choices that are globally suboptimal in this regime. At 10 and 13 nodes, the training-task data provides richer per-cluster execution profiles, and WSH's informed greedy approach converges to near-optimal solutions faster than the PSO swarm can explore the enlarged space.

C. Why the Energy-Aware Scheduler Wins on Epigenomics

Epigenomics' compute-intensive, fully parallel structure creates many independent branches. Each branch competes for the highest-power node C1. The energy score function (Eq. 13) with $\gamma = 0.22$ effectively spreads branches across C1, C2, and C3, reducing C1's bottleneck load and improving concurrency—an inadvertent but measurable makespan benefit.

D. Communication–Energy Trade-off

Tables 8 and 9 reveal a clear trade-off: schedulers that spread work across subclusters (WSH, PSO, GA, Energy) incur higher communication but much lower energy than

HEFT. For workloads where intermediate data sizes are small relative to computation time (Gene2Life BLAST stage), the communication penalty is negligible. For workloads with large intermediate files (Epigenomics FASTQ stages), the PSO scheduler's explicit communication minimisation becomes important and reduces C_{comm} by 41% relative to WSH.

E. Current Limitations

(i) **Modelled vs. measured communication.** Actual packet latency depends on physical hardware and network contention that Docker CPU pinning cannot replicate. The $L_s = 4\,\text{ms}$ and $L_c = 18\,\text{ms}$ constants are engineering estimates; real deployments may differ significantly.

(ii) **Modelled vs. measured energy.** RAPL or external watt-meter instrumentation was not available. Power values are derived from vCPU counts and memory; real TDP values may differ, especially under memory-bandwidth stress.

(iii) **Single-server emulation.** All 12 containers share one physical machine; true inter-node network variability, NUMA effects, and memory contention across containers are not fully captured.

(iv) **Three workflows.** While these are well-established scientific workflow benchmarks [11], they may not represent all DAG topologies (e.g., highly irregular fan-in/fan-out, very deep pipelines, or workflows with non-uniform edge weights).

(v) **PSO/GA overhead.** The 504 and 832 objective evaluations required by PSO and GA respectively add scheduling overhead of 5–10% relative to WSH. For very short workflows, this overhead may outweigh the makespan gain.

F. Future Directions

- Validate with real multi-node physical hardware and RAPL/watt-meter instrumentation for ground-truth energy measurement.
- Extend evaluation to public cloud (AWS EC2, GCP) to validate at production scale with true network variability.
- Investigate deep reinforcement learning schedulers [6] on the same benchmark suite for a fair comparison with learned policies.
- Add deadline-aware scheduling to handle SLA constraints alongside makespan and energy, enabling WSH++ for commercial cloud deployments.
- Explore multi-objective Pareto-front optimisation (NSGA-II) to expose the full makespan–energy trade-off curve rather than a single weighted objective.
- Scale to larger workflows ($n > 500$ jobs) common in genomics pipelines to assess how PSO/GA overhead scales relative to makespan savings.

VII. Conclusion

This paper presented **WSH++**, a comprehensive extension of the Workflow Scheduler on Hadoop (WSH) that systematically addresses all three core limitations identified by Rahmani et al. (2025). The communication-aware cost model provides a principled, physics-grounded foundation for scheduling in distributed storage environments where inter-node data transfer is a first-class concern. The PSO scheduler demonstrates that global search substantially improves on the greedy WSH heuristic—winning 8 of 15 workflow–node configurations and achieving up to 6.8% additional makespan reduction on the large-fan-out Avianflu benchmark. The GA scheduler provides a complementary evolutionary search with comparable quality. The energy-aware scheduler reduces predicted energy consumption by 60–80% relative to HEFT across all three workflows, making heterogeneous Hadoop scheduling compatible with green computing objectives.

Beyond the algorithmic contributions, WSH++ provides a complete, reproducible evaluation harness: a Java 17 implementation with Docker CPU pinning, 75 deterministically seeded scheduler runs, and open-source code. The benchmark suite closely reproduces the original paper's results (within 2%), validating both the implementation and the extended schedulers.

Taken together, these contributions advance the state of the art in heterogeneous Hadoop workflow scheduling from a single-objective greedy heuristic with zeroed communication costs to a multi-objective framework that jointly optimises makespan, communication, and energy—a practical foundation for the next generation of sustainable large-scale data processing systems.

Acknowledgements

The authors gratefully acknowledge the guidance and mentorship of **Dr. Srinivas Naik**, Associate Professor, Department of Computer Science and Engineering, IIITDM Kurnool, whose expert supervision shaped the direction of this research. We also thank the IIITDM Kurnool HPC

facility for access to the computing resources used in this work.

References

[1] A. M. Rahmani, E. Y. Chamzini, M. Pourshaban, and M. Hosseinzadeh, "Scheduling of big data workflows in the Hadoop framework with heterogeneous computing cluster," *Arabian J. Sci. Eng.*, vol. 50, pp. 12449–12461, 2025.

[2] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, 2002.

[3] Z. Tang, M. Liu, A. Ammar, K. Li, and K. Li, "An optimized MapReduce workflow scheduling algorithm for heterogeneous computing," *J. Supercomput.*, vol. 72, no. 6, pp. 2059–2079, 2016.

[4] M. Bux, J. Brandt, C. Witt, J. Dowling, and U. Leser, "Hi-WAY: Execution of scientific workflows on Hadoop YARN," in *Proc. EDBT*, 2017, pp. 668–679.

[5] H. Mikram, S. El Kafhali, and Y. Saadi, "HEPGA: A new effective hybrid algorithm for scientific workflow scheduling in cloud computing," *Simul. Model. Pract. Theory*, 2023.

[6] J. Xue, T. Wang, and P. Cai, "Towards efficient workflow scheduling over YARN cluster using deep reinforcement learning," in *Proc. IEEE GLOBECOM*, 2023, pp. 473–478.

[7] L. Yang, Y. Xia, X. Zhang, L. Ye, and Y. Zhan, "Classification-based diverse workflows scheduling in clouds," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 1, pp. 630–641, 2022.

[8] J. Meshkati and F. Safi-Esfahani, "Energy-aware resource utilization based on particle swarm optimization and artificial bee colony algorithms in cloud computing," *J. Supercomput.*, vol. 75, no. 5, pp. 2455–2496, 2019.

[9] C. Hanen, "Study of a NP-hard cyclic scheduling problem," *Eur. J. Oper. Res.*, vol. 72, no. 1, pp. 82–101, 1994.

[10] S. Wang, Y. Duan, Y. Lei, P. Du, and Y. Wang, "Electricity-cost-aware multi-workflow scheduling in heterogeneous cloud," *Computing*, 2024.

[11] G. Juve, A. Chervenak, E. Deelman, S. Bharathi, G. Mehta, and K. Vahi, "Characterizing and profiling scientific workflows," *Future Gener. Comput. Syst.*, vol. 29, no. 3, pp. 682–692, 2013.

[12] Apache Software Foundation, "Apache Hadoop," Version 3.4.3, 2023. [Online]. Available: https://hadoop.apache.org